

2024

Adapted from Thayne
Breetzke

INFORMATION SYSTEMS



STRUCTURED QUERY LANGUAGE
SQL NOTES

STRUCTURED QUERY LANGUAGE (SQL)

ADDITIONAL NOTES

1.1 Introduction

SQL (Structured Query Language) is a powerful structured language used for manipulating databases in relational database management systems (RDMSs). SQL statements fall into two broad categories: **Data Definition Language (DDL)** statements and **Data Manipulation Language (DML)** statements. DDL statements are used to manipulate the structure of a database. For example, they allow you to create objects such as databases, tables, and stored procedures. There are two types of DML query statements: an **action query** statement and a **select query** statement. Select queries are used to retrieve data from a database, while action queries are used to modify data in a database (i.e. action queries allow records to be inserted, updated, and deleted.)

Because SQL is used in almost all relational database systems (Microsoft SQL Server, Oracle, Microsoft Access, etc), you will be able to apply the knowledge you gain here in almost any other relational database environment you might encounter in the future.

These notes focus on DML statements defined by ANSI SQL. ANSI SQL defines the standard SQL syntax that provides the foundation for all other versions of SQL.

1.2 Retrieving Data with SQL

To retrieve data from a database, we use a SELECT statement. It has the following syntax:

```
SELECT select_list
    [FROM table_name [JOIN STATEMENT table2_name ...]]
    [WHERE criteria]
    [GROUP BY columns]
    [HAVING criteria]
    [ORDER BY columns]
```

The SELECT statement returns a set of records called a **view** or a **result set**.

NOTE:

Use uppercase letters for SQL clause commands (e.g. SELECT and FROM), use lowercase letters for database objects, and place each command on a new line.

Simple SELECT Statements

An asterisk (*) can be used as a wildcard symbol to request all columns in the tables in the FROM clause, as shown below:

```
SELECT *
FROM Authors
```

This returns all columns in all rows in the **Authors** table.

To restrict the columns returned, the names of the required columns can be specified, as follows:

```
SELECT au_fname, au_lname  
FROM Authors
```

The columns can be renamed (i.e. aliases can be created) in the result set, as follows:

```
SELECT au_fname AS firstname, au_lname AS lastname  
FROM Authors
```

NOTE:

In certain versions of SQL the AS keyword does not have to be included to create an alias, in which case the following would be a valid statement:

```
SELECT au_fname firstname, au_lname lastname  
FROM Authors
```

The following query concatenates two columns into a new column:

```
SELECT au_fname & ' ' & au_lname AS name  
FROM Authors
```

The standard arithmetic operators (+, -, *, /, %) can be used to manipulate numeric data. The standard operator precedence rules apply (i.e. BODMAS - brackets, of, division, multiplication, addition and subtraction.)

The following statement creates a new column that is the quantity value plus 10 percent:

```
SELECT ord_num, (qty * 1.1) AS newquant  
FROM Sales
```

Using Comparison and Logical Operators

The following table lists the comparison operators that can be used in WHERE clauses:

Operator	Meaning
=	Equal To
<> or !=	Not Equal To
>	Greater Than
!>	Not Greater Than
>=	Greater Than or Equal To
<	Less Than
!<	Not Less Than
<=	Less Than or Equal To
LIKE	

Valid Comparison Operators

The following logical operators can be used in WHERE clauses (in order of precedence):

Operator	Meaning
NOT	Inverse of the expression (e.g. Not(True) = False)
AND	Both expressions must be True
OR	At least one of the two expressions must be True
IN(Field List)	
NOT IN(Field List)	

Valid Logical Operators

Note that comparison operators (>, <, <> etc) have a higher order of precedence than logical operators. (AND, NOT, OR etc). An example of a statement using comparison and logical operators is given below:

```
SELECT Name, StateProv, City
FROM Publishers
WHERE StateProv = 'CA' AND City <> 'Senoma'
```

NOTE:

String field values must be enclosed in single quotation marks. When the data type is a date or time, you must enclose the value in hash signs. No special characters surround numeric values.

Using Data Restrictions

You can use the WHERE clause to return only the rows that meet the WHERE clause criteria. The following example returns only the rows where **au_lname** contains the value "**Smith**":

```
SELECT *
FROM Authors
WHERE au_lname = 'Smith'
```

The following example returns only the rows where **HireDate** contains a date later than 22/03/2001:

```
SELECT *
FROM Payroll
WHERE HireDate > '22/03/2001'    ' some SQL servers use #22/03/2001#
```

Using Ranges

There are two ways to handle ranges: either by using multiple comparison operators or by using the BETWEEN operator. The following statements illustrate the two methods, each returning the same results:

```
SELECT *
FROM Sales
WHERE qty >= 10
OR qty <= 50

SELECT *
FROM Sales
WHERE qty BETWEEN 10 AND 50
```

Note that the values listed in the BETWEEN clause (10 and 15 in the above example) are included in the result set. (i.e. BETWEEN is the same as >= AND <=)

Using Lists and Subqueries

You can select from a table based on possible matches from a list of items by using either a series of OR operators, or by preceding the list with the IN operator. The following statements execute identically:

```
SELECT *  
  FROM Authors  
 WHERE au_Iname = 'Smith'  
        OR au_Iname = 'Jones'
```

```
SELECT *  
  FROM Authors  
 WHERE au_Iname IN ('Smith', 'Jones')
```

Using the IN operator has the following advantages:

- The query is much easier to read and maintain.
- It provides a much easier way of managing nested SELECT statements (**subqueries**).

Subqueries can be used anywhere that an expression is expected, but they can also present a list of values to be evaluated by the IN operator. The following query returns a set of records that list all of the authors that have a mailing address in the same city as one of the stores in the database:

This SELECT is executed first

```
SELECT *  
  FROM Authors  
 WHERE city IN (SELECT city FROM Stores)
```



The inner query is executed first and returns a list of all cities. The list is then used to restrict the list of authors returned by the outer query.

Nested SELECT statements can be used to great effect. The following SQL statement for an Employment Agency in town lists all the jobs on their books that they do NOT have any applicants for! i.e. A skill is required for the job, but none of the applicants on their books can supply the skills required:

```
SELECT SkillCode, SkillDescription  
  FROM tblSkill  
 WHERE SkillCode NOT IN  
        (SELECT SkillCode FROM tblAppSkill)
```

PLEASE NOTE!! Only ONE field can be returned in the “inner” nested SELECT statement when the IN or NOT IN keyword is used. If you really want to return more than one field in the “inner” nested SELECT statement, you have to make use of the EXISTS or NOT EXISTS keyword. When EXISTS or NOT EXISTS is used, you HAVE to also supply the joining mechanism between the two tables concerned. The SQL statement below only calculates the LineTotal for Archived Invoices, if the StockID sold is still sold today. (i.e. The StockID exists in our current version of the Stock table, tblStock)

```
SELECT (InvPrice * InvQty) AS LineTotal  
  FROM tblArchiveInvoiceLine  
 WHERE EXISTS  
        (SELECT * FROM tblStock  
         WHERE tblStock.StockID = tblArchiveInvoiceLine.StockID)
```

Approximate Searches

SQL provides a set of wildcards and a special operator called LIKE to specify approximate search criteria (i.e. to return all rows where a column's contents is similar to a specified pattern.) The following table shows the wildcard characters valid for the Microsoft Access implementation of SQL:

Character	Use
*	Matches any string of zero or more characters
?	Matches any single character
[a-b]	Defines a group of characters from <i>a</i> to <i>b</i> , any one of which matches a single character in that position
[!a-b]	Defines a group of characters from <i>a</i> to <i>b</i> , all of which are excluded from representing a single character in that position
#	Matches any single numeric character

SQL Wildcard Characters (MS Access)

The following table shows the wildcard characters valid for Microsoft SQL Server's implementation of SQL:

Character	Use
%	Matches any string of zero or more characters
_ (underscore)	Matches any single character
[a-b]	Defines a group of characters from <i>a</i> to <i>b</i> , any one of which matches a single character in that position
[^a-b]	Defines a group of characters from <i>a</i> to <i>b</i> , all of which are excluded from representing a single character in that position

SQL Wildcard Characters (MS SQL Server & Jet)

Given the pattern [S-T]?i*, the following values would match: "**This**", "**String**", and "**Thin**". The following values would not: "**The**", and "**Then**".

Examine the following examples that make use of the LIKE operator:

```
SELECT *
  FROM Authors
 WHERE au_lname LIKE 'S%'           Access: 'S*'
```

The above returns all rows where the author's last name begins with the letter **S**.

```
SELECT *
  FROM Authors
 WHERE au_lname LIKE '_a_'         Access: '?a?'
```

The above returns all rows where the author's last name is three letters long having the letter **A** as the middle character.

```
SELECT *
  FROM Authors
 WHERE au_lname LIKE 'M[^c]%'      Access: 'M[!c]*'
```

The above returns all rows where the author's last name begins with the letter **M** and does not have the letter **C** as the second character.

Searching Using Nulls

Null means “unknown” or “undefined”. For example, a database that tracks order dates and ship dates in an order table may use a null value for a ship date to indicate an order that has not yet been shipped. You can search for null values by using IS NULL:

```
SELECT *  
  FROM Order  
 WHERE shipdate IS NULL
```

NOTE:

Never use “= NULL” since nothing can equal an undefined value.

Sorting Data

SQL provides mechanisms for sorting and grouping data returned by a SELECT statement. The ORDER BY clause together with an ASC or DESC keyword after each field arranges the result set in a specified order (ASC is an abbreviation for “ascending”, and DESC is an abbreviation for “descending”). If ASC or DESC is not supplied, ascending order is assumed. The following example sorts the results of the authors query by last name:

```
SELECT *  
  FROM Authors  
 WHERE au_state = 'CA'  
 ORDER BY au_lname ASC
```

You can enter more than one field in the ORDER BY clause as shown in the example below. The example sorts the records by **City** (ascending) withascin **StateProv** (descending).

```
SELECT StateProv, City  
  FROM Publishers  
 ORDER BY StateProv DESC, City ASC
```

The DISTINCT Clause

There are times when you select data from a table that has more than one occurrence of the rows you are trying to collect. For example, imagine that you wish to get a list of all the customers that have at least one order on file in the **Orders** table. The problem is that some customers have several orders in the table. You do not want to see those names appear more than once in your result set. You can use the DISTINCT keyword to make sure that you do not get duplicates of the same customer in your result set. An example is shown below:

```
SELECT DISTINCT au_ID  
  FROM Titles  
 ORDER BY au_ID
```

If you include more than one column in the SELECT list, all columns are used to evaluate the uniqueness of the row. **PLEASE NOTE!!** The fields selected are CRITICAL, as they are ALL used to determine whether the contents of the ENTIRE row are duplicated in another row: ONLY in this case (when EVERYTHING is the same in two rows) will DISTINCT “kick in”: If just one field is different, DISTINCT will select the two rows separately.

```
SELECT DISTINCT Title, Units  
  FROM BookSales
```

Using Aggregate Functions

Aggregate functions allow you to perform summary functions on data, such as summation or counting. The following table provides a list of aggregate functions and their uses:

Aggregate function	Description
SUM(column)	Returns the total of all the non-null values in a column
AVG(column)	Returns the average value of all non-null values in a column
COUNT(column)	Returns the total number of non-null values in a column
MIN(column)	Returns the lowest value from a column
MAX(column)	Returns the highest value from a column

Aggregate Functions

The following example counts the number of publishers that have a state location listed in the database:

```
SELECT COUNT(state) AS CountState
FROM Publishers
```

To count all the publishers, regardless of whether they have a state listed, you could do the following:

```
SELECT COUNT(*) AS NumPublishers
FROM Publishers
```

In cases like the above it would be better if we could give each computed result a name. This can be done using the AS keyword, as shown in the following example:

```
SELECT COUNT(Units) AS UnitCount,
       AVG(Units) AS UnitAvg,
       SUM(Units) AS UnitSum,
       MIN(Units) AS UnitMin,
       MAX(Units) AS UnitMax
FROM BookSales
WHERE Qtr = 1
```

You can also do Math at the same time as in:

```
SELECT SUM(Price * Qty) As InvTotal
FROM tblInvoiceLine
```

1.3 Correlating Data with WHERE Statements (p 78)

In the previous examples, the SQL statements affected only one table. You can use WHERE statements to define relationships between columns in different tables, thus allowing you to select data in multiple tables at the same time. For example, the following query finds all book sales including the author ID (the sales table does not include the **au_id** column; the tables are linked with the **title_id** field):

```
SELECT sales.* TitleAuthor.au_id
FROM Sales, TitleAuthor
WHERE Sales.title_id = TitleAuthor.title_id
```

Note that the field names are prefixed with the table name to which they belong.

1.4 Modifying Data with SQL (p 76)

SQL statements can be used to update and modify data in existing tables. There are three modification statements in SQL: INSERT, UPDATE, and DELETE.

Inserting Data

The INSERT statement inserts new data into the database (one row at a time). The statement has the following syntax:

```
INSERT INTO table_name [field_list]
VALUES (value_list)
```

An example of an INSERT statement is given below:

```
INSERT INTO Stores
VALUES ('2076', 'My Book Store', '32 East Way', 'Raytown', 'UT',
'84502')
```

Note that the values must be in the same order as the columns in the table. You can also create a new row in the table without entering a value for every column. As long as the columns allow nulls or have default values associated with them enforced by the underlying data engine, you can provide partial column list as follows:

```
INSERT INTO Stores (stor_id, stor_name)
VALUES ('2078', 'Your Book Store')
```

A separate INSERT statement must be used for each row you want to add to a table.

If any field is an **AutoNumber** field, this must NOT be included in the VALUES list. (i.e. if there is an AutoNumber field in the table, the field list MUST be specified (omitting the AutoNumber field from the list)).

NOTE:

Microsoft Access SQL requires the use of the INTO keyword, all other databases simply require INSERT as shown below:

```
INSERT stores (stor_id, stor_name)
VALUES ('2078', 'Your Book Store')
```

Referential Integrity: You need to first add a record on the “one” side of a relationship first / make sure that it exists, before any associated “many” records can be added. (Otherwise the records on the “many” side of the relationship would be orphaned, violating Referential Integrity).

Updating Data

The UPDATE statement makes modifications to existing data, and has the following syntax:

```
UPDATE table_name
SET field_name = value [,field_name = value ...]
[WHERE criteria]
```

Consider the following example:

```
UPDATE Authors
SET au_lname = 'Smith'
```

The above changes ALL the authors' last names to **“Smith”**. Normally, you would want to make changes more selectively. The WHERE clause allows for this, as shown below:

```
UPDATE Authors
SET au_fname = 'Al'
```

```
WHERE au_lname = 'Ringer'
      AND au_fname = 'Albert'
```

Note that multiple SET clauses can be used in one UPDATE statement, as shown below:

```
UPDATE Customers
  SET CustType = 'RETAIL',
      CustDisc = 10,
      CustDate = #15/01/2001#
  WHERE CustSurname = 'Ringer'
      AND CustFirstName = 'Albert'
```

NOTE: An UPDATE statement without a WHERE is **very** dangerous!

Deleting Data

The DELETE statement deletes data from a database, from a single record to an entire table. The syntax is shown below:

```
DELETE FROM table_name
  [WHERE criteria]
```

Consider the following example:

```
DELETE FROM Authors
```

The above would delete all the data from the **Authors** table. To selectively delete data, a WHERE clause must be used, as follows:

```
DELETE FROM Authors
  WHERE au_lname = 'Ringer'
```

NOTE:

Microsoft Access SQL requires the use of the FROM keyword, whereas other databases simply require a DELETE as shown below:

```
DELETE Authors
  WHERE au_lname = 'Ringer'
```

In these notes you learned how to create SQL statements that select data from existing tables, and examined a variety of optional clauses that you can add to the SELECT statement to refine the result set returned by the statement. You learned how to join multiple tables together using the JOIN clause, and how to combine the results of multiple SQL queries into a single result set using the UNION clause.

You also learned how to add, delete, and modify data within tables using the INSERT, DELETE and UPDATE statements. You also learned how to copy data from one table to another by using the INSERT INTO FROM version of the INSERT statement.

It should be clear that SQL is a powerful manipulation language that carries many benefits to you as a software developer. For this reason, it has become the de facto standard language for manipulating database information in today's world.

References

Amundsen, M & Smith, C. (1999). **Teach Yourself Database Programming with Visual Basic 5 in 21 Days**. SAMS Publishing, U.S.A.

Ekedahl, M. (2000). **MCSD Guide to Developing Desktop Application with Microsoft Visual Basic 6.0: Advanced Topics**. Course Technology, Canada.

Lee, M. (1999). **MCSD: Visual Basic 6 Distributed Applications**. SYBEX Inc, U.S.A.

Kochar, N and Kramer, D (1992). **Introduction to Oracle: SQL and PL/SQL Using Procedure Builder**
Volume Two – Participant Guide



Name & Surname: _____

1

After completing this exercise, please indicate your level of understanding for the content covered by this exercise.

Tick one option:



I understand.



I do not understand.

Question:

Show a list of the names, surnames and positions of all employees.

SQL Statement:

2

After completing this exercise, please indicate your level of understanding for the content covered by this exercise.

Tick one option:



I understand.



I do not understand.

Question:

Show all details for employees earning more than R20 000.

SQL Statement:

3

After completing this exercise, please indicate your level of understanding for the content covered by this exercise.

Tick one option:



I understand.



I do not understand.

Question:

Show the names, surnames, hire dates and department of employees hired on or before 1 Jan 1990.

SQL Statement:

4

After completing this exercise, please indicate your level of understanding for the content covered by this exercise.

Tick one option:



I understand.



I do not understand.

Question:

Show the names, surnames and department IDs of employees hired between 12 March 1992 and 1 Jan 1997.

SQL Statement:

5

After completing this exercise, please indicate your level of understanding for the content covered by this exercise.

Tick one option:



I understand.



I do not understand.

Question:

Show the names of all employees with names beginning with B.

SQL Statement:

6

After completing this exercise, please indicate your level of understanding for the content covered by this exercise.

Tick one option:



I understand.



I do not understand.

Question:

Show the names of all employees with 5 letter names beginning with C.

SQL Statement:

7

After completing this exercise, please indicate your level of understanding for the content covered by this exercise.

Tick one option:



I understand.



I do not understand.

Question:

Show the names, surnames of all employees earning more than R18 000.

SQL Statement:

8

After completing this exercise, please indicate your level of understanding for the content covered by this exercise.

Tick one option:



I understand.



I do not understand.

Question:

Show the surnames, names and salaries of all employees in sequence of their name.

SQL Statement:

9

After completing this exercise, please indicate your level of understanding for the content covered by this exercise.

Tick one option:



I understand.



I do not understand.

Question:

Output the number of employees in the company.

SQL Statement:

10

After completing this exercise, please indicate your level of understanding for the content covered by this exercise.

Tick one option:



I understand.



I do not understand.

Question:

Find out the highest salary.

SQL Statement: